



Object Oriented System Analysis And Design

Group Assignment

Group Members

1.Redeit Aragie	UUR00225/17
2.Nathnael Yirga	UUR00295/17
3.Abenezer Abebe	UUR01368/16
4.Aboneh Tenkir	UUR00590/16
5.Abrham Mekonen	UUR01404/16

AI-Based Drone Management System for Delivery

Introduction

In the modern digital era, technological advancements have significantly transformed how organizations operate and deliver services. Emerging technologies such as artificial intelligence (AI), automation, robotics, and data analytics are increasingly being used to improve efficiency, accuracy, and decision-making. These technologies are playing a crucial role in solving complex real-world problems and enabling smarter systems across various industries.

One of the most rapidly growing technologies is the use of **Unmanned Aerial Vehicles (UAVs)**, commonly known as drones. Drones are aircraft that operate without a human pilot onboard and can be controlled remotely or autonomously using software systems. Initially developed for military use, drones are now widely used in civilian and commercial applications such as agriculture, surveillance, environmental monitoring, construction, disaster management, and especially **delivery services**.

Drone delivery systems are becoming popular because they offer fast, efficient, and cost-effective transportation of goods. Companies can use drones to deliver packages, medical supplies, and essential goods to remote or hard-to-reach areas. This is particularly useful in emergency situations, urban traffic congestion, and rural regions where traditional delivery methods are slow or unavailable.

However, as the number of drones used for delivery increases, managing their operations becomes more complex. Organizations need to coordinate multiple drones, schedule deliveries, monitor flight status, manage battery usage, and ensure timely and safe delivery of packages. Traditional manual management methods are not sufficient to handle such complex operations, especially when multiple drones are operating simultaneously.

Another challenge is ensuring real-time monitoring and quick response to issues such as low battery, route deviation, or delivery failure. Without an intelligent system, these problems can lead to delays, inefficiency, and increased operational risks. Additionally, drones generate large amounts of data such as delivery logs, flight paths, and sensor data, which must be properly stored and analyzed.

To overcome these challenges, integrating **Artificial Intelligence (AI)** into drone management systems provides a powerful solution. AI enables systems to analyze data, make predictions, automate decisions, and optimize operations. For example, AI can predict battery failure, optimize delivery routes, detect anomalies, and provide real-time alerts.

An **AI-Based Drone Management System for Delivery** is designed to manage, monitor, and optimize drone delivery operations using intelligent technologies. It provides a centralized platform for scheduling deliveries, monitoring drones in real time, managing fleets, and analyzing data for better decision-making.

This project focuses on designing and developing such a system using simulation, as real drones are not available. The system will demonstrate how AI can improve delivery efficiency, safety, and operational management.

Statement of the Problem

The increasing use of drones in delivery services has introduced new opportunities for fast and efficient transportation of goods. However, managing multiple drones for delivery operations presents several challenges.

Currently, many drone delivery operations rely on manual control or basic systems that lack intelligent coordination and centralized management. This makes it difficult to monitor drone activities, schedule deliveries efficiently, and ensure timely package delivery.

The main problem is the lack of an **intelligent and centralized system** that can manage drone delivery operations effectively using AI. This problem affects organizations and users involved in delivery services, especially where multiple drones are used.

The problem occurs in environments where drone delivery is required, such as urban areas, rural regions, and emergency situations. It arises due to the absence of systems that can provide real-time monitoring, predictive analysis, and automated decision-making.

If not solved, this problem can lead to delays, increased operational costs, inefficient delivery, and safety risks. Therefore, there is a need to develop an **AI-Based Drone Management System for Delivery** that improves efficiency, monitoring, and decision-making.

Objectives

General Objective

Our general objective of AI-Based Drone Management System for Delivery is to design and develop an intelligent system that improves the efficiency, monitoring, and management of drone-based delivery operations.

Specific Objectives

- To develop a centralized system for managing drone delivery operations
- To implement real-time monitoring of delivery drones
- To design intelligent delivery scheduling
- To manage drone fleets effectively

- To provide alert notifications for delivery issues
- To integrate AI for predictive maintenance and decision-making

Scope of the Project

1. Geographical Scope

The system is limited to a specific organization or local area where drone delivery operations are simulated and managed. It does not cover global or large-scale deployment.

2. Technical Scope

- Software-based system developed using **Java**
- Desktop-based interface using **JavaFX**
- Drone operations are simulated instead of using real drones
- AI-based logic is implemented for prediction and analysis (rule-based simulation)
- Database integration using **MySQL**
- Supports basic delivery management and monitoring features

Significance of the Project

The **AI-Based Drone Management System for Delivery** is important because it addresses real-world challenges in modern delivery systems. As demand for fast and efficient delivery services increases, traditional transportation methods face limitations such as traffic congestion, delays, and high operational costs.

This project improves delivery efficiency by enabling automated scheduling and real-time monitoring of drone deliveries. It reduces human effort and minimizes errors through intelligent decision-making supported by AI. The system enhances safety by providing instant alerts for critical situations such as low battery, route deviation, or delivery failure.

Additionally, the project supports better data management by organizing delivery information such as routes, delivery time, and drone performance. This helps organizations analyze operations and improve future delivery planning. For students, this project provides practical knowledge in AI, system development, and simulation, making it valuable academically and technically.

Feasibility of the Project

The feasibility study evaluates whether the **AI-Based Drone Management System for Delivery** can be successfully developed and implemented within the given constraints. This project is

analyzed in terms of technical, economic, operational, organizational, schedule, and legal feasibility.

1. Technical Feasibility

The project is technically feasible because it can be developed using available and well-known technologies such as Python, web frameworks (Flask/Django), and AI libraries (TensorFlow, Scikit-learn). Since real drones are not required, the system uses simulation to represent drone delivery operations, which reduces complexity while still demonstrating system functionality. The development tools and platforms are accessible and suitable for student-level implementation.

2. Economic Feasibility

The project is economically feasible as it requires minimal cost. Most of the software tools and libraries used are free and open-source. The main expenses are limited to internet access, documentation, and minor materials. Since drone operations are simulated, there is no need for expensive hardware, making the project affordable for students.

3. Operational Feasibility

The system is operationally feasible because it is designed to be simple, user-friendly, and efficient. Users such as administrators and operators can easily interact with the system to manage deliveries, monitor drones, and receive alerts. The system improves operational efficiency by automating delivery scheduling and monitoring processes.

4. Organizational Feasibility

The project is organizationally feasible as it aligns with the capabilities and resources of the intended users. It does not require major organizational changes or specialized infrastructure. The system can be easily adopted in small-scale delivery environments or academic settings. Additionally, users can operate the system with minimal training due to its intuitive design and clear functionalities.

5. Schedule Feasibility

The project is schedule feasible because it can be completed within the allocated academic timeframe. By following a structured development approach such as the System Development Life Cycle (SDLC), all phases including analysis, design, implementation, and testing can be completed within the planned schedule.

6. Legal Feasibility

The project is legally feasible as it does not violate any laws or regulations. Since the system uses simulation instead of real drones, it avoids legal restrictions related to drone usage. Additionally, the system ensures proper handling and protection of data.

Methodology

The development of the **AI-Based Drone Management System for Delivery** follows a structured approach:

1. Preliminary Investigation and Introduction

At this stage, the problem of managing drone delivery operations is identified. The need for an AI-based system to improve delivery efficiency, monitoring, and coordination is clearly defined.

2. Analysis

The system requirements are analyzed based on delivery operations. This includes identifying how drones will deliver packages, how users interact with the system, and what data needs to be collected (such as delivery routes, time, and drone status).

3. Design

The system is designed to include key modules such as delivery scheduling, drone monitoring dashboard, fleet management, and AI analysis. The database structure is also designed to store delivery data, drone information, and user details. Simulation logic for drone movement and delivery tasks is planned.

4. Implementation

The system is developed using programming tools such as Python and web frameworks. Features such as delivery scheduling, real-time monitoring, alert notifications, and AI-based prediction are implemented. Drone behavior is simulated to represent delivery operations.

5. Testing

- **Unit Testing:** Each module (e.g., scheduling, monitoring, alerts) is tested individually to ensure it works correctly.
- **System Testing:** The entire system is tested as a whole by simulating delivery operations and verifying that all components work together properly, including alerts and real-time updates.

6. Maintenance

After testing, the system is improved by fixing errors, optimizing performance, and updating features based on feedback. This ensures the system remains reliable and efficient.

Data Collection Method (Preferred Method)

For this project, the preferred data collection method is **observation and practical simulation of delivery scenarios**.

This method is chosen because the project focuses on drone delivery operations, and simulation allows direct understanding of how deliveries are scheduled, monitored, and managed. By observing simulated delivery processes, it becomes easier to identify system requirements such as real-time tracking, alert notifications, and route management.

Software and Hardware Tools

For this project, the **AI-Based Drone Management System for Delivery** will be developed and tested using **software-based drone simulation** instead of real drones. This approach is appropriate for a student-level project and allows full testing of system functionalities such as delivery scheduling, monitoring, and AI-based analysis.

1. Software Tools

Tool/Software	Purpose
Java	Used as the main programming language to develop the AI-Based Drone Management System.
JavaFX	Used to design the graphical user interface (GUI) including login, dashboard, and management pages.
Apache NetBeans	Used as the development environment for coding, debugging, and running the JavaFX application.
SQLite / MySQL	Used to store system data such as drones, delivery tasks, and alerts.
JavaFX TableView	Used to display drone data, tasks, and system information in a structured table format.
Java Simulation Logic	Used to simulate drone behavior such as battery level changes, task progress, and alert generation.

2. Hardware Tools

Hardware	Purpose
Laptop / Desktop	Used to develop, run, and test the system and simulation (minimum 8GB RAM recommended).
Internet / Network Connection	Used for downloading tools, running simulations, and enabling system communication.

Hardware	Purpose
Storage Devices (HDD/SSD)	Used to store simulated delivery data, logs, and system outputs.

Simulation Note

Due to the unavailability of real drones, this project uses **software simulation** to represent drone delivery operations. The simulation mimics real-world drone behavior such as movement, delivery tasks, battery consumption, and status updates. This allows all system modules—including scheduling, monitoring, fleet management, and AI analysis—to be tested effectively in a controlled environment.

Project Budget

The **AI-Based Drone Management System for Delivery** is developed as a student-level project. Therefore, the budget is minimal and mainly covers basic operational and documentation costs. Most tools used are free and open-source.

Item	Description	Cost (ETB)	Remarks
Laptop / Desktop	Personal computer used for development	0	Already available
Internet Access	Data for downloading tools and running simulations	500	Estimated
Software Tools	Python, IDEs, frameworks, databases	0	Open-source
AI Libraries	TensorFlow, Scikit-learn, etc.	0	Open-source
Storage (Optional)	External storage or cloud usage	500	Optional

Total Estimated Budget: ≈ 1,000ETB

Budget Note

No cost is allocated for drones because the system uses **simulation instead of real hardware**, making the project cost-effective and suitable for academic purposes.

Project Schedule

The project will be completed within **12 weeks**, following structured development phases.

Phase	Activities	Duration	Timeline
1. Preliminary Investigation & Introduction	Define problem, objectives, feasibility	1 week	Week 1
2. Analysis	Gather requirements and system needs	2 weeks	Week 2–3
3. Design	Design system architecture, database, and UI	2 weeks	Week 4–5
4. Implementation	Develop system modules and AI features	4 weeks	Week 6–9
5. Testing	Unit testing and system testing	2 weeks	Week 10–11
6. Maintenance & Documentation	Final improvements and report preparation	1 week	Week 12

Schedule Notes

- **Unit Testing** is performed after developing each module.
- **System Testing** is conducted after integrating all components.
- Simulation is used to test delivery operations, real-time monitoring, and alert notifications.
- The schedule is flexible depending on project progress.

Requirements and Requirement Gathering

Requirements are gathered based on delivery system needs, focusing on scheduling, monitoring, fleet coordination, and AI-based decision-making.

Functional Requirements with Measurement

Requirement	Measurement
Real-time monitoring	Updates every second

Requirement	Measurement
Alert notification	Alert generated within seconds
Fleet management	Multiple drones handled in simulation
Autonomous delivery	Drone follows delivery path
Delivery scheduling	Correct assignment of delivery tasks
Data storage	Data stored and retrieved correctly
AI analysis	Predict delivery issues accurately

Non-Functional Requirements with Measurement

Requirement	Measurement
Performance	Fast response time (within seconds)
Security	Role-based access control
Reliability	No system failure during simulation
Usability	Easy user interaction
Scalability	Handles increasing delivery tasks

Deliverable Requirements

The deliverable requirements define the **expected outputs** of the **AI-Based Drone Management System for Delivery** project. These deliverables represent the final results that will be produced to demonstrate that the system meets its objectives and requirements.

1. System Deliverables

1. **AI-Based Drone Management System for Delivery (Software Application)**
 - A fully functional system that supports delivery scheduling, real-time monitoring, fleet management, alert notifications, and AI-based analysis using simulated drones.
2. **Drone Simulation Module**
 - A simulation environment that mimics drone delivery operations such as movement, battery usage, and delivery tasks.
3. **Database System**
 - A structured database that stores drone information, delivery logs, user data, and system records.
4. **AI Module**
 - Implementation of AI features such as predictive maintenance, anomaly detection, and intelligent decision support.
5. **User Interface (UI)**
 - A user-friendly interface for managing delivery operations, monitoring drones, and viewing reports.

2. Documentation Deliverables

1. **Requirement Specification Document**
 - Contains functional and non-functional requirements of the system.
2. **System Design Document**
 - Includes system architecture, diagrams (use case, DFD, ER diagram), and design details.
3. **Testing Report**
 - Includes results of unit testing and system testing, along with identified errors and fixes.
4. **User Manual**
 - Instructions on how to use the system, including system features and operations.
5. **Final Project Report**
 - Complete documentation of the project from introduction to conclusion.

3. Additional Deliverables

1. **Data Visualization and Reports**
 - Charts and dashboards showing delivery performance, drone status, and AI predictions.
2. **Project Presentation Slides**
 - Slides prepared for project defense or classroom presentation.

Requirement Elicitation and Analysis

1 Requirement Elicitation

Requirement elicitation is the process of gathering information about what the system should do from users and other sources.

For this project, requirements are identified based on **delivery operations and simulation scenarios**. The system focuses on managing drone-based delivery activities such as scheduling, monitoring, and alert handling.

The main needs identified include:

- Managing multiple delivery drones
- Scheduling delivery tasks efficiently
- Monitoring drone activities in real time
- Generating alerts for critical events (e.g., low battery)
- Storing and analyzing delivery data
- Supporting decision-making using AI

These requirements are derived from observing how delivery systems work and adapting them into a simulated environment.

13.2 Requirement Analysis (System Model)

Requirement analysis involves organizing and structuring the collected requirements into a clear system model.

System Model Description

The **AI-Based Drone Management System for Delivery** is designed as a centralized system that manages drone delivery operations through simulation.

Main Actors

- **Administrator** → Manages system and users
- **Operator** → Schedules deliveries and monitors drones
- **System (AI Module)** → Performs analysis and predictions

Core System Functions

- Drone registration and management
- Delivery scheduling
- Real-time monitoring
- Alert notification (in seconds)
- Fleet management
- Data collection and storage
- AI-based analysis

System Flow (Simple)

1. User logs into the system
2. Operator schedules a delivery

3. Simulated drone performs delivery
4. System monitors drone in real time
5. Alerts are generated if issues occur
6. Data is stored and analyzed

Requirement Specification

1 User Requirements

User requirements describe what users expect from the system in simple terms.

- The system should allow users to manage drone deliveries easily
- The system should display real-time delivery status
- The system should send alerts for delivery problems
- The system should allow scheduling and tracking of deliveries
- The system should support multiple drones (fleet management)
- The system should provide simple and user-friendly interfaces

2 System Requirements

System requirements describe detailed functionalities the system must provide.

Functional Requirements

- Register and manage drones
- Schedule and manage delivery tasks
- Simulate autonomous drone delivery
- Monitor drone status in real time
- Generate alert notifications within seconds
- Manage multiple drones (fleet management)
- Store delivery and drone data
- Perform AI-based analysis and prediction

Non-Functional Requirements

- Fast system response (within seconds)
- Secure login and role-based access
- Reliable system operation without failure
- Easy-to-use interface
- Ability to handle multiple delivery tasks

Requirement Validation

Requirement validation ensures that the defined requirements are correct, complete, and meet user needs.

For this project, validation will be performed using the following methods:

1. Review and Verification

All requirements will be reviewed to ensure they are clear, consistent, and aligned with the project objectives.

2. Testing with Simulation

The system will be tested using simulated drone delivery scenarios to verify that all requirements are implemented correctly.

3. User Feedback

Test users (students or instructors) will interact with the system to confirm that it is easy to use and meets expectations.

4. Requirement Documentation

All requirements will be properly documented in the project report, including:

- Functional requirements
- Non-functional requirements
- Requirement measurements
- System behavior

5. Validation Criteria

The system will be considered valid if:

- All required features are implemented
- The system performs correctly during simulation
- Alerts and monitoring work in real time
- Users can successfully perform delivery operations

The 6 P's of the AI-Based Drone Management System for Delivery

1. Purpose

The purpose of the AI-Based Drone Management System for Delivery is to design a smart and efficient system that improves delivery operations using AI-based decision-making, real-time monitoring, and automated drone management. The system aims to enhance delivery speed, accuracy, and reliability through simulation-based drone control.

2. Participants

The main participants involved in the system are:

- **Administrator** – Manages the overall system, users, and drones
- **Operator/User** – Schedules deliveries, monitors drone activities, and receives alerts
- **AI System Module** – Provides intelligent decision-making such as prediction and anomaly detection
- **Instructor/Client (Academic Supervisor)** – Evaluates and guides the project development

3. Principles

The system is based on the following principles:

- Automation of drone delivery processes
- Real-time monitoring and control
- AI-based decision-making and prediction
- Data-driven analysis for improved performance
- Scalability for multiple drone (fleet) management
- Simulation-based testing instead of real drones

4. Product

The product is an **AI-Based Drone Management System for Delivery**, which is a software-based platform that:

- Simulates drone delivery operations
- Manages drone fleets and delivery tasks
- Provides real-time monitoring dashboards
- Generates alert notifications in seconds
- Stores and analyzes delivery data using AI

5. Place

The system is developed and used in a **simulated academic environment**. Since real drones are not available, all drone operations are simulated using software tools in a computer-based environment. The project is intended for educational purposes in a university or college setting.

6. Process

The development process follows the System Development Life Cycle (SDLC):

- **Preliminary Investigation & Introduction** – Identify problem and define objectives
- **Analysis** – Gather and analyze system requirements
- **Design** – Create system architecture, database, and UI design
- **Implementation** – Develop system using Python, AI libraries, and web framework

- **Testing** – Perform unit testing and system testing using simulation
- **Maintenance** – Fix errors and improve system performance after testing

Requirement Checking (Validation Criteria)

Requirement checking ensures that the collected requirements are correct, clear, and suitable for system development. The requirements of the AI-Based Drone Management System for Delivery are evaluated based on the following criteria:

1. Validity

The requirements are considered valid because they accurately reflect the real needs of a drone delivery system, such as delivery scheduling, real-time monitoring, fleet management, and alert notifications. Even though the system is simulation-based, the requirements still represent realistic drone delivery operations.

2. Consistency

The requirements are consistent because there are no contradictions between functional and non-functional requirements. For example, real-time monitoring, alert notifications, and data storage all support the same goal of efficient drone delivery management without conflict.

3. Completeness

The requirements are complete because they cover all major functionalities needed for the system, including drone registration, delivery scheduling, autonomous flight simulation, monitoring, AI analysis, and data storage. Both user and system requirements are fully addressed.

4. Realism (Feasibility)

The requirements are realistic because they are suitable for a student-level project and can be implemented using available technologies such as Python, Flask/Django, and AI libraries. Since real drones are not used, simulation makes the system practical and achievable within time and resource limits.

5. Verifiability

The requirements are verifiable because each requirement can be tested through simulation. For example:

- Delivery scheduling can be tested by creating simulated missions
- Real-time monitoring can be verified by tracking drone status updates
- Alert notifications can be tested by triggering simulated faults
- AI predictions can be evaluated using test datasets

Prioritization of Requirements

Requirement prioritization helps to classify system requirements based on their importance for system success and implementation feasibility. The requirements of the **AI-Based Drone Management System for Delivery** are prioritized as follows:

1. High Priority (Must Have)

These are the most critical requirements that the system cannot function without:

- Drone registration and management
- Delivery scheduling system
- Real-time drone monitoring
- Fleet management (multiple drones control)
- Data collection and storage (flight logs, delivery records)
- Basic alert notification system (e.g., failure, low battery simulation)
- User authentication (admin/operator login)

These features form the **core functionality** of the system.

2. Medium Priority (Should Have)

These are important but the system can still run without them in a basic form:

- AI-based analysis (prediction of delivery performance or delays)
- Advanced alert system with improved intelligence
- Dashboard visualization (graphs, charts for monitoring data)
- Performance reporting and analytics
- Improved real-time tracking accuracy

These features improve **efficiency and intelligence** of the system.

3. Low Priority (Nice to Have)

These are optional features that can be added if time allows:

- Advanced machine learning optimization models
- Voice-based control or commands
- Mobile application version of the system
- Advanced simulation of weather conditions affecting drones
- Integration with external delivery APIs

System Analysis

Activities of System Analysis (After Requirement Gathering)

After collecting requirements, system analysis focuses on understanding, organizing, and refining those requirements to prepare for system design.

1. Requirement Classification and Structuring

The gathered requirements are grouped into:

- Functional requirements (e.g., drone scheduling, monitoring, alerts)
- Non-functional requirements (e.g., performance, security, reliability)
- User and system requirements

This helps to organize the system scope clearly.

2. Feasibility Refinement

The initial feasibility study is reviewed in detail:

- Technical feasibility (Python, AI tools, simulation)
- Economic feasibility (low cost, student budget)
- Operational feasibility (easy for users to operate)
- Organizational feasibility (suitable for academic use)
- Legal feasibility (no real drone operation, simulation only)

3. System Modeling

The system is represented in abstract models to understand how it works:

- Use case model (actors and interactions)
- Data Flow Diagram (DFD)
- High-level system architecture
- Process flow of drone delivery operations

4. Data Flow and Process Analysis

- Identifying how data moves in the system
- Drone data → system → AI module → user dashboard
- Understanding inputs, processes, and outputs

Example:

- Input: delivery request, drone status
- Process: scheduling, AI analysis, monitoring
- Output: alerts, reports, delivery status

5. Problem Decomposition

The whole system is broken into smaller modules:

- Drone management module
- Delivery scheduling module
- Monitoring module
- AI analysis module
- Alert system module

This makes development easier and structured.

6. System Constraints Identification

- No real drones (simulation only)
- Limited computing resources (student laptop)
- Time constraints (academic schedule)
- Limited budget

7. Validation of Requirements

- Checking completeness, consistency, and correctness
- Ensuring requirements match user needs
- Removing ambiguity and duplication

Actors and Use Cases – AI-Based Drone Management System for Delivery

1. Actors

Actors represent the users or external systems that interact with the AI-Based Drone Management System for Delivery.

1. Administrator

- Has full control over the system
- Manages users, drones, and system settings

2. Operator / User

- Uses the system to schedule and monitor drone deliveries
- Interacts with dashboards and receives alerts

3. AI Module (System Component)

- Performs intelligent analysis and decision-making
- Detects anomalies and predicts issues

4. Drone Simulator (External System)

- Simulates drone behavior such as flight, movement, and battery usage
- Sends status updates to the system

2. Use Cases

Use cases describe the main functions performed by the system.

A. Administrator Use Cases

- Login to system
- Manage users (add/remove/update operators)
- Register and manage drones
- Monitor all drone activities
- View system reports and logs
- Configure system settings

B. Operator / User Use Cases

- Login to system
- Schedule delivery missions
- Assign drones to delivery tasks
- Monitor real-time drone status
- Receive alert notifications (delay, failure, low battery)
- View delivery history and reports

C. AI Module Use Cases

- Analyze drone performance data
- Predict possible failures or delays
- Detect anomalies in drone behavior
- Generate intelligent recommendations

D. Drone Simulator Use Cases

- Simulate drone takeoff and landing
- Simulate flight path execution
- Send real-time status updates
- Simulate battery consumption
- Report delivery completion or failure

3. Summary

The system has **4 main actors (Administrator, Operator, AI Module, and Drone Simulator)** and they interact with key use cases such as:

- Drone management
- Delivery scheduling
- Real-time monitoring
- AI prediction and analysis
- Alert notification system
- Simulation of drone operations

Objects and Classes – AI-Based Drone Management System for Delivery

In object-oriented design, the system is broken into **classes (blueprints)** and **objects (real instances)** to represent real-world entities in the drone delivery system.

1. Identified Classes

1. User Class

Represents all system users (Admin, Operator)

Attributes:

- userID
- name
- email
- password
- role (Admin / Operator)

Methods:

- login()
- logout()
- viewDashboard()

2. Drone Class

Represents a delivery drone in the system

Attributes:

- droneID

- status (idle, in-flight, charging, failed)
- batteryLevel
- location
- speed

Methods:

- takeOff()
- land()
- updateStatus()
- reportBattery()

3. DeliveryTask Class

Represents a delivery mission

Attributes:

- taskID
- sourceLocation
- destinationLocation
- packageInfo
- assignedDrone
- status

Methods:

- assignDrone()
- startDelivery()
- completeDelivery()
- cancelTask()

4. FleetManager Class

Manages all drones in the system

Attributes:

- fleetID
- listOfDrones

Methods:

- addDrone()
- removeDrone()
- assignDroneToTask()
- monitorFleet()

5. AIAnalysis Class

Handles intelligent decision-making

Attributes:

- dataset
- modelAccuracy

Methods:

- predictDelay()
- detectAnomaly()
- optimizeRoute()

6. Alert Class

Handles system notifications

Attributes:

- alertID
- alertType (battery, failure, delay)
- message
- timestamp

Methods:

- generateAlert()
- sendNotification()

7. SimulationEngine Class

Simulates drone behavior (since no real drones are used)

Attributes:

- simulationSpeed
- environmentData

Methods:

- simulateFlight()
- simulateBatteryDrain()
- simulateWeatherImpact()

2. Objects

- User → “Admin1”, “OperatorA”
- Drone → “Drone001”, “Drone002”
- DeliveryTask → “Task101”
- Alert → “LowBatteryAlert1”
- FleetManager → “MainFleet”
- AIAnalysis → “PredictionModelV1”
- SimulationEngine → “TestEnvironment”

Relationships Between Classes – AI-Based Drone Management System for Delivery

In the AI-Based Drone Management System for Delivery, classes are connected through different types of relationships such as **association**, **aggregation**, and **dependency**. These relationships show how the system components interact with each other.

1. User – DeliveryTask (Association)

- A **User** (Operator/Admin) creates and manages **DeliveryTasks**.
- One user can create multiple delivery tasks.
- Each delivery task is created and controlled by a user.

Relationship:

User **1** → **many** DeliveryTask

2. DeliveryTask – Drone (Association)

- A **DeliveryTask** is assigned to a **Drone** for execution.
- One drone can perform multiple tasks over time, but one task is assigned to one drone at a time.

Relationship:

Drone **1** → **many** DeliveryTask (over time)

DeliveryTask **1** → **1** Drone (at a time)

3. FleetManager – Drone (Aggregation)

- **FleetManager** manages a group of drones.
- Drones exist independently but are managed as a fleet.

Relationship:

FleetManager **1** → **many** Drone (Aggregation)

4. AIAnalysis – Drone (Dependency)

- The **AIAnalysis** class uses drone data (battery level, location, status).
- It depends on drone information for prediction and anomaly detection.

Relationship:

AIAnalysis → Drone (Dependency)

5. AIAnalysis – DeliveryTask (Dependency)

- AI uses delivery task data to predict delays and optimize delivery routes.

Relationship:

AIAnalysis → DeliveryTask (Dependency)

6. Drone – Alert (Dependency)

- A **Drone** generates data that can trigger an **Alert** (e.g., low battery, failure).
- Alerts depend on drone status.

Relationship:

Drone → Alert (Dependency)

7. SimulationEngine – All Classes (Dependency)

- The **SimulationEngine** simulates the behavior of drones, tasks, and alerts.
- It interacts with all system classes during testing.

Relationship:

SimulationEngine → Drone, DeliveryTask, Alert (Dependency)

8. User – Admin / Operator (Inheritance)

Admin and Operator inherit common properties and behaviors from the User class such as login, logout, and user information management.

Relationship:

User → Admin, Operator (Inheritance)

9. FleetManager – Drone (Composition)

FleetManager strongly owns and manages multiple drones. Drones are part of the fleet and cannot exist within the system without being managed by the FleetManager.

Relationship:

FleetManager ♦→ Drone (Composition)

10. DeliveryTask – Alert (Composition)

Alerts are generated only within a delivery task to indicate issues such as delay or low battery. Without a delivery task, alerts do not exist in the system.

Relationship:

DeliveryTask ♦→ Alert (Composition)

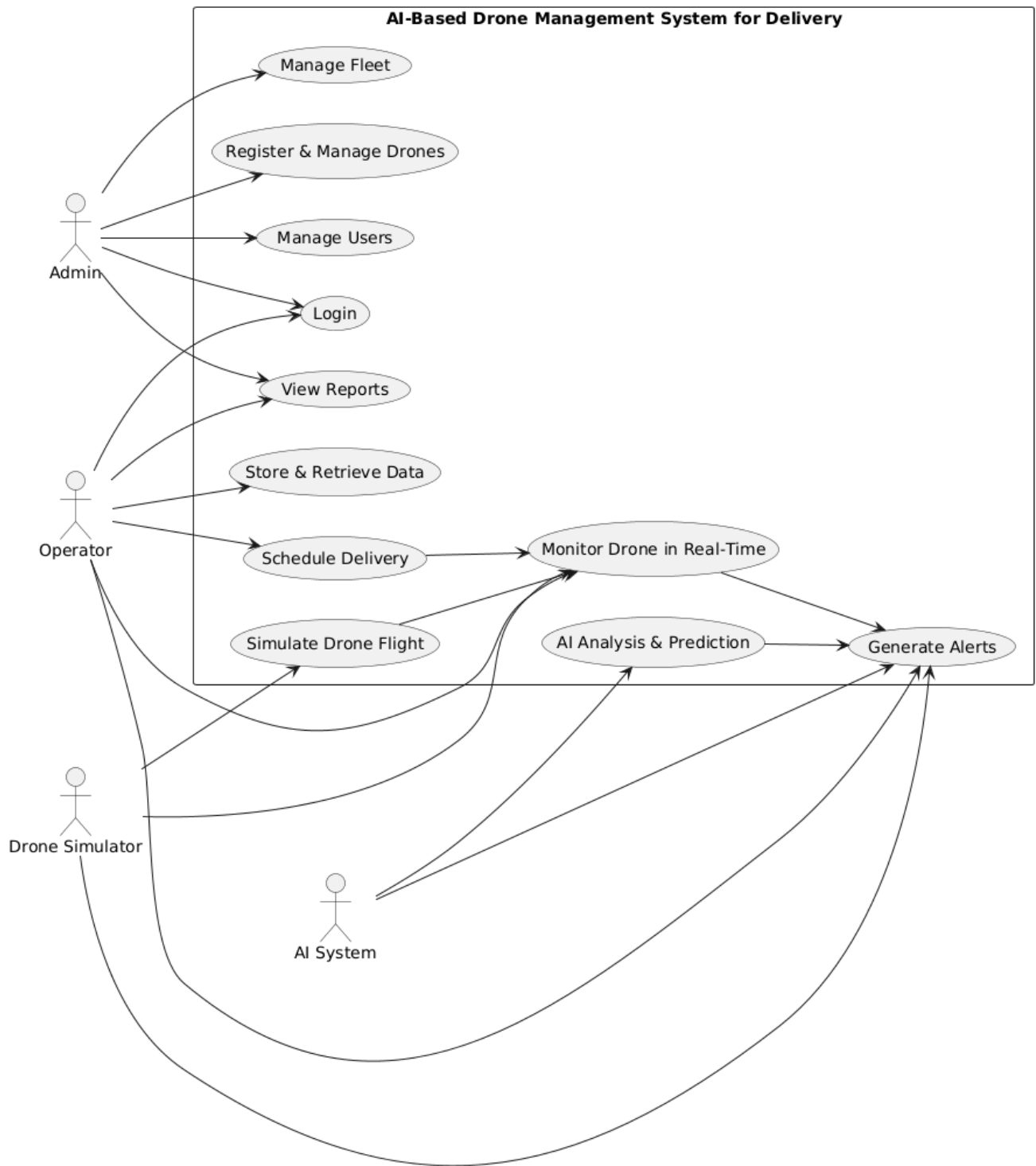
Summary of Relationships

Classes	Relationship Type
User – DeliveryTask	Association
Drone – DeliveryTask	Association
FleetManager – Drone	Aggregation
AIAnalysis – Drone	Dependency
AIAnalysis – DeliveryTask	Dependency
Drone – Alert	Dependency
SimulationEngine – System Classes	Dependency
User – Admin / Operator	Inheritance
FleetManager – Drone	Composition
DeliveryTask – Alert	Composition

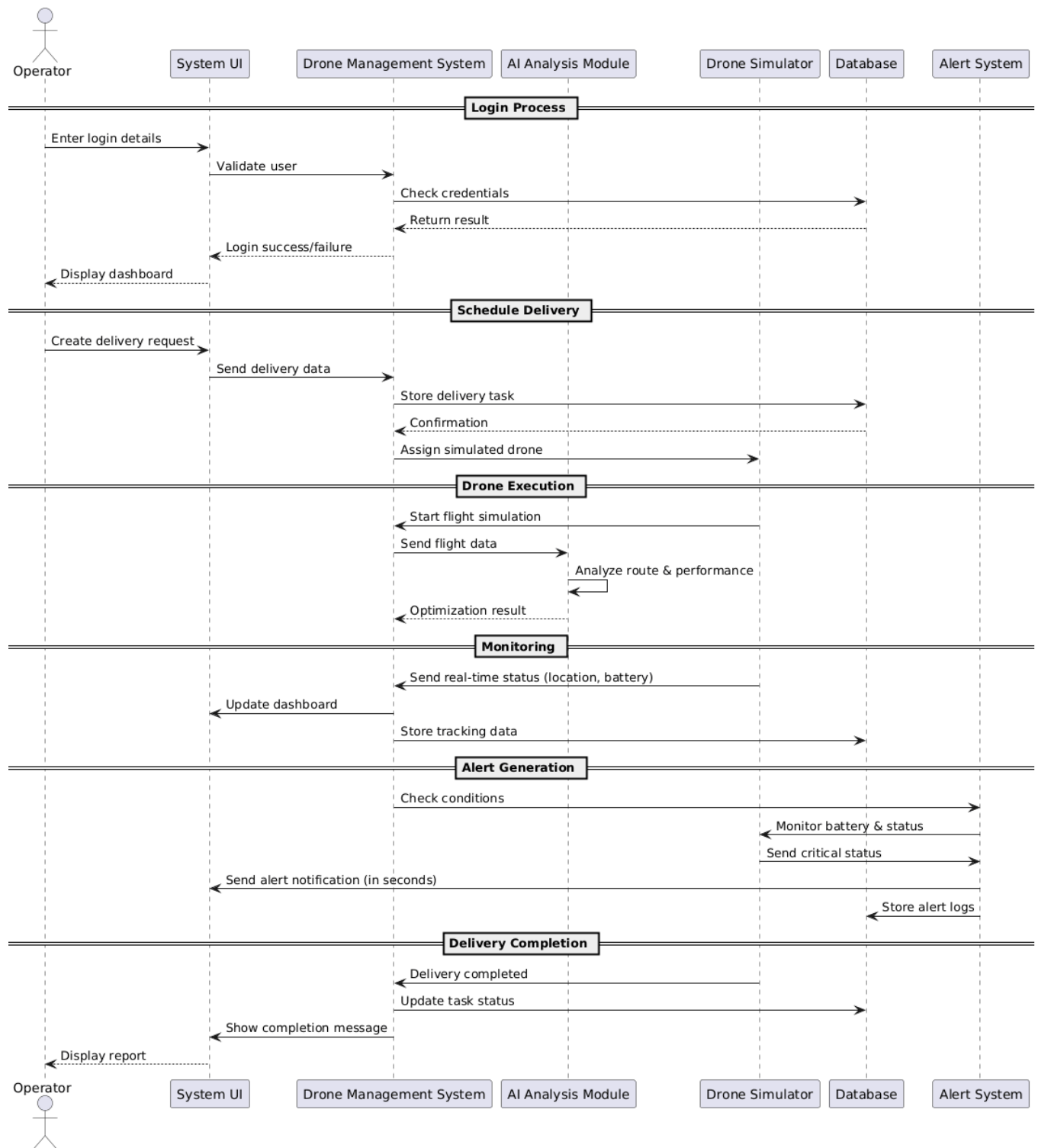
Final Note

These relationships ensure that the system is well-structured, where:

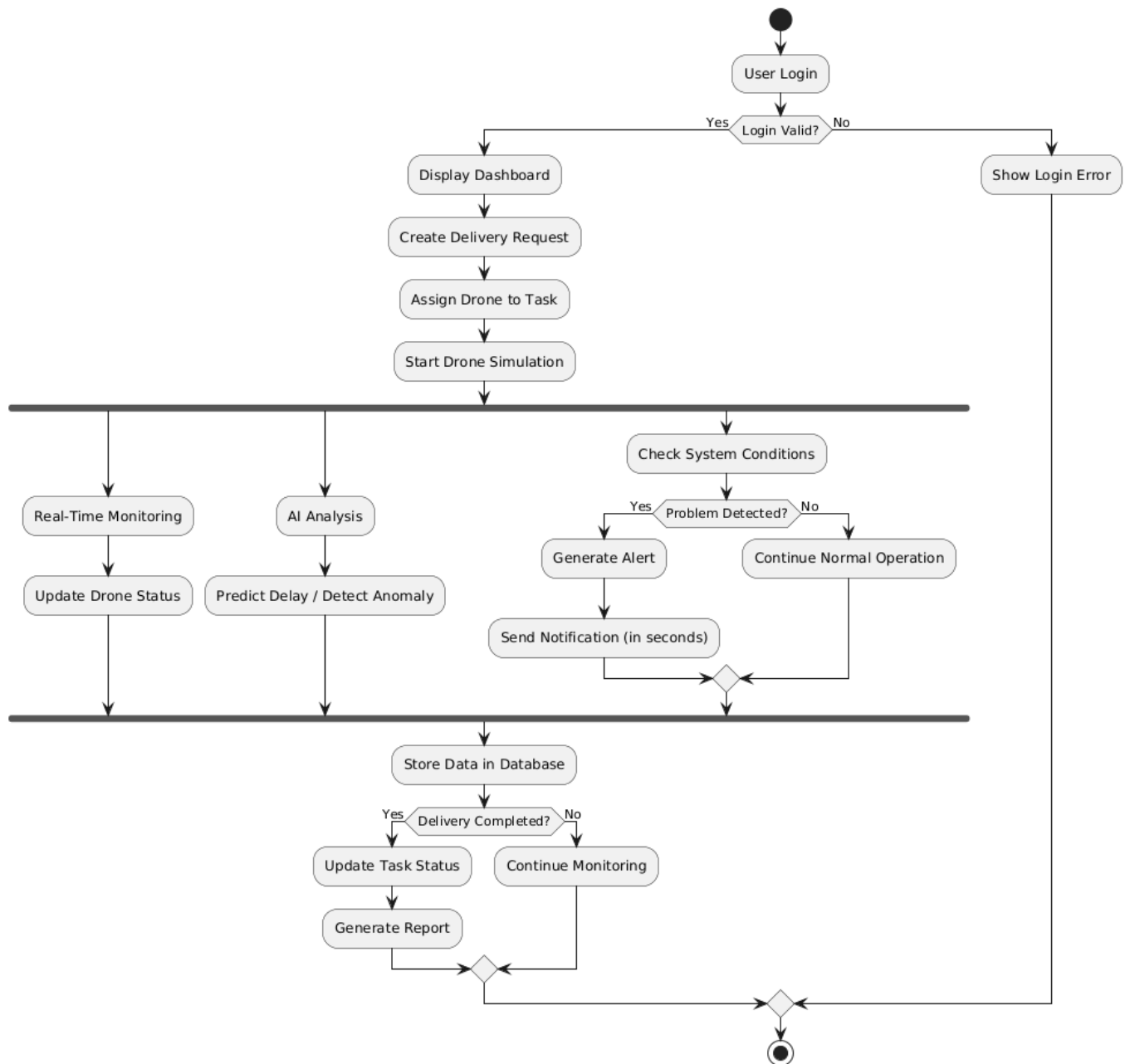
- Users manage tasks
- Drones execute deliveries
- AI analyzes performance
- FleetManager organizes drones
- SimulationEngine tests the system



Use case diagram



Sequence diagram



Activity diagram

Reviewing Analysis Model

Reviewing the analysis model is the process of evaluating all system analysis outputs to ensure they are accurate, complete, consistent, and aligned with the project objectives before moving to the design and implementation phases.

For the AI-Based Drone Management System for Delivery, the developed analysis models were reviewed as follows:

1. Requirement Review

The functional and non-functional requirements were checked to confirm that they address the main needs of the system, including delivery scheduling, real-time monitoring, fleet management, AI analysis, and alert notification.

2. Use Case Model Review

The use case diagram was reviewed to ensure that all actors (Administrator, Operator, AI Module, and Drone Simulator) are correctly identified and connected with the appropriate system functions.

3. Structural Model Review

The class diagram and object diagram were reviewed to verify that all important classes, attributes, methods, and relationships (association, inheritance, composition, dependency) are correctly represented.

4. Behavioral Model Review

The sequence diagram, activity diagram, state diagram, and collaboration diagram were reviewed to ensure that system workflows, message passing, and state changes are logically correct.

5. Consistency Check

All models were compared to ensure consistency. For example:

- Use cases match system requirements
- Sequence and activity diagrams reflect real workflows
- Class diagram supports required functionalities

6. Completeness Check

The analysis model was checked to confirm that no major system component or function was omitted.

7. Feasibility and Practicality Review

The models were reviewed to ensure they are realistic for a student-level project using simulation instead of real drones.

Conclusion

The analysis models for the AI-Based Drone Management System for Delivery are valid, consistent, complete, and suitable to proceed to the system design and implementation stages.

SYSTEM DESIGN

System Design Goals

The design of the AI-Based Drone Management System for Delivery is guided by the following goals:

- **Reliability:** The system should perform delivery operations accurately and consistently.
- **Efficiency:** Task assignment, monitoring, and notifications should be processed quickly.
- **Maintainability:** The system should be easy to modify, debug, and upgrade.
- **Scalability:** More drones, users, and delivery tasks can be added when needed.
- **Usability:** The user interface should be simple and understandable.
- **Security:** Access to system resources should be controlled through authentication.
- **Flexibility:** The design should allow future integration with real drones and external systems.
- **Reusability:** Components and classes should be reusable in future projects.
- **Cost-effectiveness:** The project should use low-cost and open-source tools suitable for students.

System Design Process

The system design process followed the steps below:

1. Review the analysis models and confirmed system requirements.
2. Identify the major subsystems of the application.
3. Select a suitable software architecture for implementation.
4. Refine analysis classes into design classes with methods and relationships.
5. Design interfaces for users and communication between components.
6. Design the database structure for storing system data.
7. Define deployment of software and hardware resources.
8. Review the design for correctness, completeness, consistency, and feasibility.

Architectural Design

The system uses a **Three-Tier Architecture** because it separates user interaction, business logic, and data management.

1. Presentation Layer

This layer provides interfaces for administrators and operators. Users log in, register drones, schedule deliveries, monitor tasks, and view reports through the web interface.

2. Business Logic Layer

This layer processes all system functions including drone management, delivery scheduling, fleet monitoring, AI analysis, simulation control, and alert generation.

3. Data Layer

This layer stores all persistent data such as users, drones, delivery tasks, alerts, logs, and reports in the database.

The selected architecture improves system organization, simplifies maintenance, and supports future expansion.

Subsystem Decomposition

The system is divided into the following subsystems:

1. User Management Subsystem

Manages login, logout, user roles, and account access.

2. Drone Management Subsystem

Registers drones and updates drone status, battery level, speed, and location.

3. Delivery Management Subsystem

Creates delivery tasks, assigns drones, tracks mission progress, and updates task completion.

4. AI Analysis Subsystem

Uses collected data to predict delays, detect anomalies, and optimize routes.

5. Notification Subsystem

Generates alerts for low battery, drone failure, route delay, and communication problems.

6. Database Subsystem

Stores and retrieves all operational and historical records.

7. Simulation Subsystem

Simulates drone flight, battery consumption, route progress, and environmental effects.

Object Design

The main system objects and classes are refined as follows:

User Class

Used for all users such as administrators and operators.
Functions: login, logout, dashboard access.

Drone Class

Represents a drone with status, battery level, location, and speed.
Functions: takeoff, land, update status, report battery.

DeliveryTask Class

Represents a delivery request with source, destination, package details, and assigned drone.
Functions: assign drone, start delivery, complete delivery, cancel task.

FleetManager Class

Manages all drones in the fleet.
Functions: add drone, remove drone, assign drone, monitor fleet.

AIAnalysis Class

Performs intelligent processing.
Functions: predict delay, detect anomaly, optimize route.

Alert Class

Creates notifications for critical events.
Functions: generate alert, send notification.

SimulationEngine Class

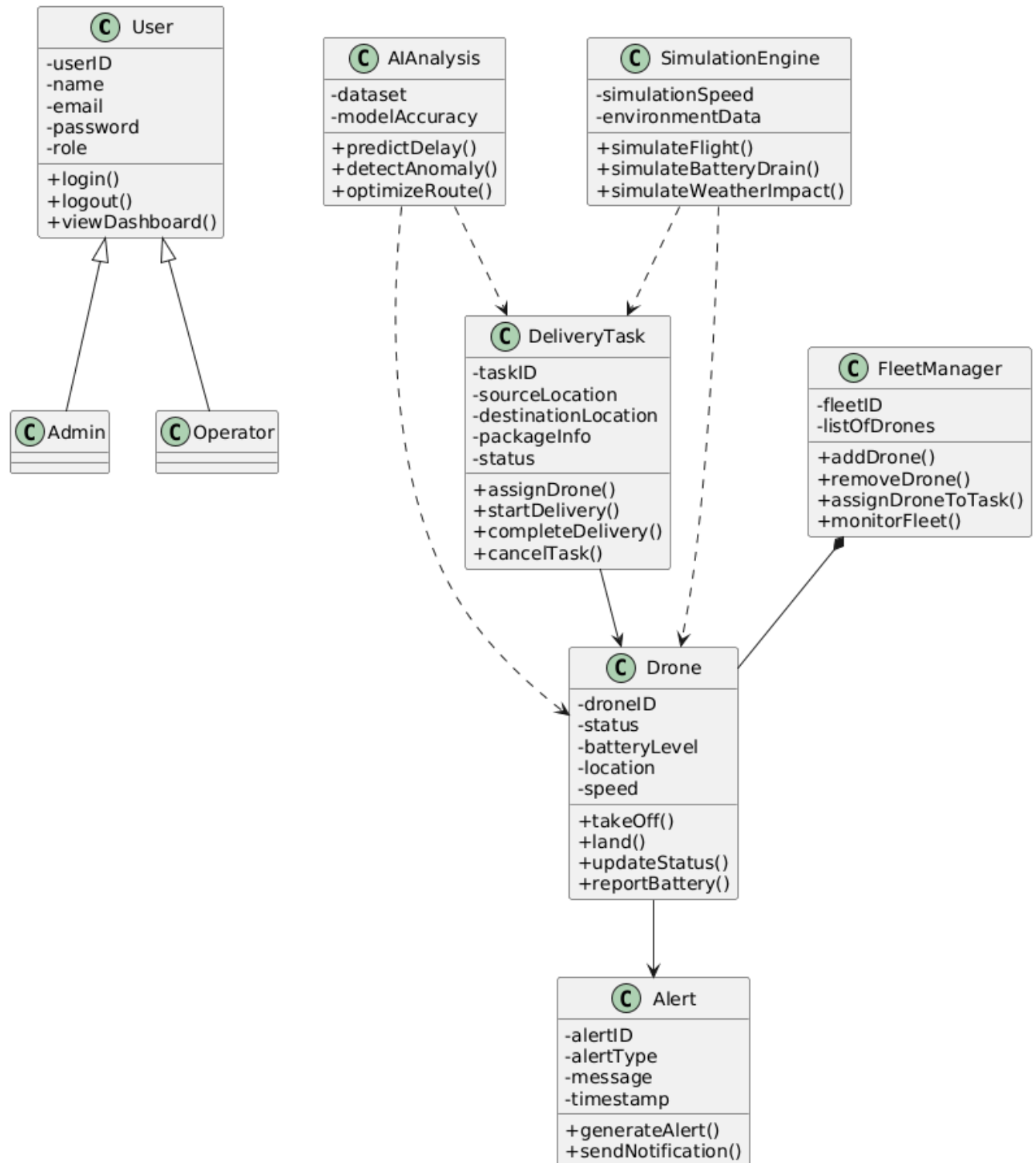
Controls simulated drone behavior.

Functions: simulate flight, battery drain, and environmental impact.

Class Diagram

The class diagram describes the static structure of the system and relationships between classes.

- **Inheritance:** Admin and Operator inherit from User.
- **Composition:** FleetManager contains multiple Drone objects.
- **Association:** DeliveryTask is linked with Drone.
- **Dependency:** AIAnalysis uses DeliveryTask and Drone data for predictions.



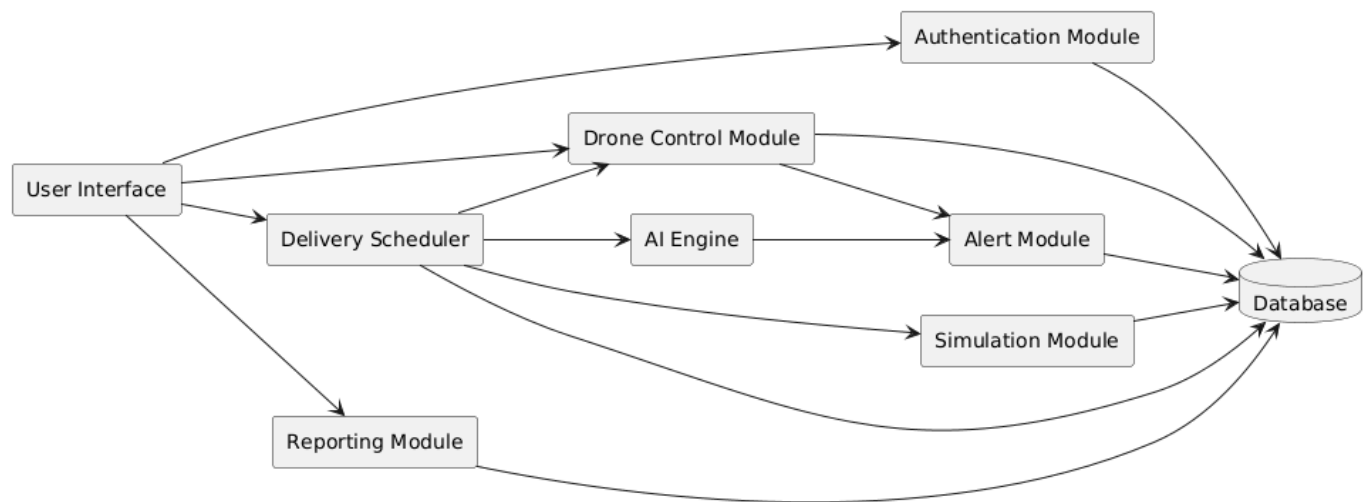
Component Design

The software is divided into components that perform specific tasks.

Main Components:

- **User Interface Component** – Handles screens and dashboards.
- **Authentication Component** – Validates users and controls access.
- **Drone Control Component** – Manages drone records and status.
- **Delivery Scheduler Component** – Creates and assigns tasks.
- **AI Engine Component** – Performs prediction and optimization.
- **Alert Component** – Sends warnings and notifications.
- **Simulation Component** – Simulates drone activities.
- **Database Access Component** – Performs data storage and retrieval.
- **Reporting Component** – Generates reports and analytics.

Each component communicates through clear interfaces, making the system modular and maintainable.



The Existing System Components

The existing system is mainly manual or uses basic tools with limited automation. The major existing components are:

1. Human Operator

Responsible for receiving delivery requests, assigning tasks manually, and tracking progress.

2. Communication Tools

Phone calls, messages, or paper records used for communication between staff and customers.

3. Manual Record Keeping

Delivery information, schedules, and reports are stored on paper or simple spreadsheets.

4. Basic Monitoring

Status of deliveries is checked manually through calls or periodic updates.

5. Traditional Transportation / Delivery Process

Package movement depends on manual transportation methods without drone support.

Deployment Design

The deployment design shows how the system will run in a real environment.

1. Client Device

Used by administrators and operators through a web browser.

2. Application Server

Hosts the web application, business logic, AI engine, and simulation engine.

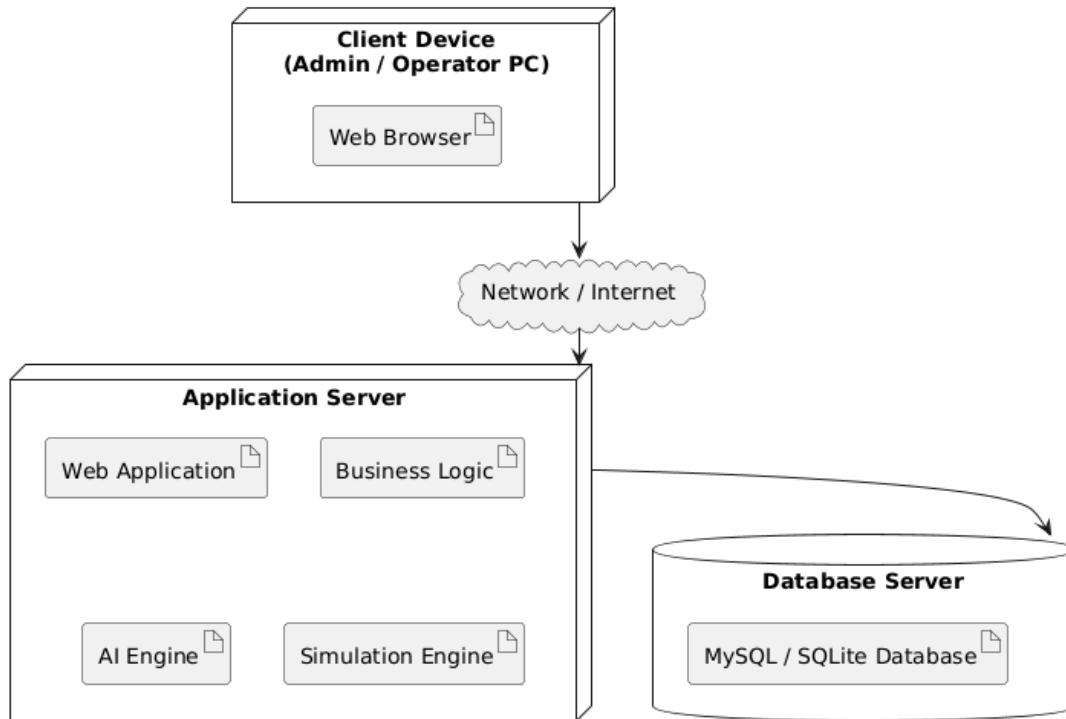
3. Database Server

Stores users, drones, tasks, alerts, logs, and reports.

4. Network

Connects users to the application and database servers.

This structure supports centralized management and remote access.



Interface Design

The system contains the following user interfaces:

1. Login Page

Allows secure user authentication.

2. Dashboard

Displays summary information such as active drones, tasks, and alerts.

3. Drone Management Page

Used to register drones and update drone information.

4. Delivery Scheduling Page

Used to create delivery requests and assign drones.

5. Monitoring Page

Shows real-time drone location, battery level, and task progress.

6. Alert Page

Displays urgent warnings and notifications.

7. Report Page

Shows delivery history, drone performance, and analytics.

Data Management Design

The database stores and organizes all system information.

Main Tables

Users

Stores userID, name, email, password, and role.

Drones

Stores droneID, status, batteryLevel, location, and speed.

DeliveryTasks

Stores taskID, sourceLocation, destinationLocation, packageInfo, assignedDrone, and status.

Alerts

Stores alertID, alertType, message, and timestamp.

Logs

Stores monitoring data, user activities, and system events.

The database design ensures consistency, fast retrieval, and secure storage.

Access Control Design

The system uses role-based access control.

Administrator Permissions

- Manage users
- Register and manage drones
- View all reports
- Configure system settings

Operator Permissions

- Create delivery tasks
- Assign drones
- Monitor deliveries
- View alerts and reports

Authentication through username and password prevents unauthorized access.

Control Flow Mechanisms

The system follows a structured operational flow:

1. User logs into the system.
2. Authentication verifies credentials.
3. Operator creates a delivery request.
4. System checks available drones.
5. Drone is assigned to the task.
6. Simulation starts the mission.
7. Monitoring updates drone status continuously.
8. AI analyzes route and predicts delays.
9. Alert system detects abnormal conditions.
10. Completion data is saved and reports are generated.

This control flow ensures smooth coordination between modules.

Boundary Conditions

The system handles exceptional and limit conditions such as:

- Invalid username or password
- No available drone for assignment
- Low battery before mission completion
- Drone communication failure
- Incorrect task input data
- Simultaneous multiple delivery requests
- Database connection error
- Unexpected system interruption

When these conditions occur, the system generates alerts, blocks invalid operations, retries processes where possible, and stores logs for troubleshooting.

Design Review

The final design was reviewed using the following criteria:

- **Correctness:** All design elements satisfy the analysis requirements.
- **Completeness:** Major modules, interfaces, and data structures are included.
- **Consistency:** Diagrams and subsystem interactions are logically connected.
- **Realism:** The design is feasible for a student-level project using simulation tools.
- **Readability:** The design is clearly organized and understandable.

The review confirms that the system is ready for implementation.

Project Summary

The **AI-Based Drone Management System for Delivery** is designed to automate and improve the process of delivery management using intelligent system features. The system provides an organized platform for managing drones, assigning delivery tasks, monitoring operations, and generating alerts in real time.

The main objective of the project is to replace manual delivery management with an efficient computerized system that enhances accuracy, speed, and decision-making. The system allows administrators and operators to register drones, schedule delivery tasks, monitor drone status, and receive alerts for critical situations such as low battery or delivery delays.

The system is developed using **JavaFX**, providing a graphical user interface that enables easy interaction with users. It includes core modules such as user authentication, drone management, task management, simulation of drone behavior, and alert notification. Since real drones are not available, the system uses simulation techniques to mimic drone operations such as battery consumption and task execution.

In addition, basic AI concepts are incorporated to support intelligent decision-making, such as identifying potential issues (e.g., low battery) and improving operational efficiency. The system also provides a dashboard for viewing statistics and system status.

Overall, the project demonstrates how modern technologies like artificial intelligence and automation can be applied to improve delivery systems. It is scalable, user-friendly, and suitable for further enhancement, such as integration with real drones, GPS tracking, and mobile applications.

References

- Artificial Intelligence Concepts. (2023). *Introduction to Artificial Intelligence*.
- Object-Oriented System Analysis and Design Notes. (2024). *Lecture Materials and Course Slides*.
- Software Engineering, Ian Sommerville. (2016). *Software Engineering (10th Edition)*. Pearson Education.

Appendix

Appendix A : Additional Use Case Scenarios

Scenario 1: Schedule Delivery

- Actor: Operator
- Description: Operator assigns a delivery task
- Outcome: Task is scheduled and drone assigned

Scenario 2: Monitor Drone

- Actor: Operator
- Description: Operator views drone status in real-time
- Outcome: System displays current drone condition

Scenario 3: Generate Alert

- Actor: AI System
- Description: System detects abnormal condition
- Outcome: Alert is generated and displayed

Appendix B: Assumptions

- Drones are simulated (no real hardware)
- Internet connection is stable
- Users are authorized (Admin/Operator roles)
- System operates in a controlled environment

Appendix C: Constraints

- No real-time GPS tracking
- Limited AI capability (rule-based)
- System designed for academic purposes
- Limited scalability

Appendix D: Glossary

Term	Meaning
Drone	Unmanned aerial vehicle used for delivery
Delivery Task	A job assigned to a drone
AI Analysis	System decision-making logic
Alert	Notification generated by system
Simulation	Artificial representation of real operations